Arquitetura Godot Engine 4.x

1. O que é a Godot Engine?

O Godot Engine é uma game engine multiplataforma repleta de recursos para criar jogos 2D e 3D a partir de uma interface unificada. Ele fornece um conjunto abrangente de ferramentas comuns para que os usuários possam simplesmente criar seus jogos sem ter que reinventar a roda. Os jogos podem ser exportados num clique para várias plataformas, incluindo as principais plataformas de desktop (Linux, macOS, Windows), plataformas móveis (Android, iOS), as baseadas na Web e também consoles. **(GODOT, 2026)**.
.

1.1. Conceitos Chave: Cenas e Nós

No coração da Godot está o sistema de Cenas e Nós (Scene/Node Tree). Uma cena é uma coleção de nós organizados em uma estrutura de árvore. Cada nó é um bloco de construção fundamental que pode representar uma funcionalidade específica, como um personagem, uma câmera, um sprite, um colisor ou um elemento de interface de usuário
.

- **Nós (Node):** São os menores blocos de construção do jogo. A Godot oferece uma vasta biblioteca de tipos de nós base, que podem ser combinados e estendidos para criar funcionalidades mais complexas. Existem nós específicos para 2D, 3D e interface de usuário
- **Cenas (Scene):** São coleções de nós que podem ser salvas e reutilizadas. Uma cena pode ser um personagem, uma arma, um menu, um nível inteiro ou qualquer outro componente do jogo. Cenas podem ser aninhadas, permitindo a criação de estruturas complexas a partir de componentes menores e modulares
- **Árvore de Cenas (SceneTree):** Todas as cenas do jogo se unem na árvore de cenas, que é a representação hierárquica de todos os nós e cenas ativos no jogo em um determinado momento
- **Sinais (Signals):** Nós emitem sinais quando eventos ocorrem. Este mecanismo permite que os nós se comuniquem de forma desacoplada, facilitando a estruturação e a manutenção do código. Por exemplo, um botão pode emitir um sinal pressed quando clicado, e outras partes do jogo podem se conectar a esse sinal para reagir ao evento

.

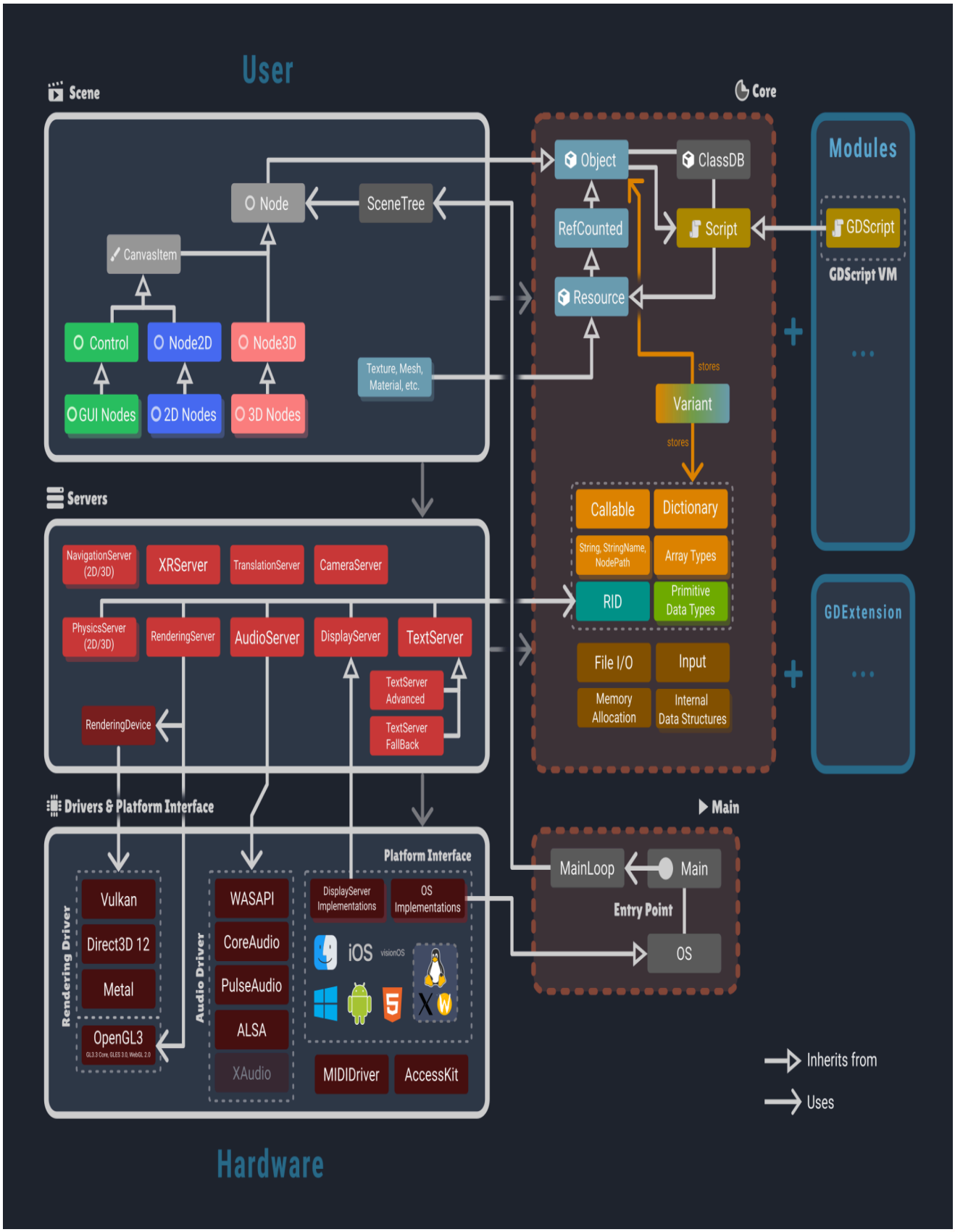
1.2. Principais Recursos

A Godot Engine 4.x oferece uma gama de recursos que a tornam uma escolha poderosa para o desenvolvimento de jogos
:

- **Editor de Cenas:** Um editor visual intuitivo para construir e organizar cenas e nós.

- **Editor de Script Integrado:** Suporte a GDScript (linguagem nativa), C# e outras linguagens via GDExtension.
- **Renderização 2D e 3D:** Capacidades avançadas de renderização, incluindo suporte a Vulkan para alta performance.
- **Ferramentas 2D**: TileMaps para design de niveis 2D baseado em tiles.
- **Física:** Motores de física 2D e 3D integrados.
- **Animação:** Ferramentas robustas para animação de sprites, esqueletos e propriedades de nós.
- **Áudio:** Sistema de áudio flexível para música e efeitos sonoros.
- **Rede:** Suporte a rede para jogos multiplayer.
- **Internacionalização:** Ferramentas para tradução e localização de jogos.
- **Sistema de Exportação:** Capacidade de exportar jogos para múltiplas plataformas com facilidade.

2. Visão Geral da Arquitetura



2.1 Camada de Cena (Scene Layer)

A camada de Cena é o nível mais alto da arquitetura do Godot, fornecendo o sistema de cenas, que é a principal forma de construir e estruturar aplicações ou jogos.

- **Árvore de Cena (Scene Tree):** Gerencia a hierarquia dos **nós (nodes)** na cena, assim como as próprias cenas dentro do Godot Engine.Os nós podem ser adicionados, acessados e removidos. Toda a árvore de cenas (e, consequentemente, a cena atual) pode ser pausada. As cenas também podem ser carregadas, trocadas e recarregadas. Também pode ser utilizada para organizar os nós em grupos e cada nó pode pertencer a mais de um grupo. Além disso a SceneTree e a implementação padrão do MainLoop utilizada pela engine, sendo a responsável pelo loop principal do jogo.
- **Nó (Node):** Os nós são os blocos de construção do Godot. Eles podem ser definidos como filhos de outro nó, formando uma estrutura em árvore. Um nó pode ter vários filhos, mas todos os nós irmãos devem ter nomes únicos. Uma árvore de nós é chamada de **cena**. As cenas podem ser salvas no disco e depois instanciadas dentro de outras cenas, o que permite grande flexibilidade na arquitetura e no modelo de dados dos projetos no Godot.

2.2 Camada de Servidores (Server Layer)

Os componentes de servidor implementam a maior parte dos subsistemas do Godot (renderização, áudio, física, etc.). Eles são objetos singleton inicializados na inicialização da engine.

2.3 Drivers / Interface de Plataforma

Essa camada abstrai detalhes de baixo nível específicos de cada plataforma, contendo drivers para APIs gráficas, sistemas de áudio e interfaces do sistema operacional (incluindo implementações específicas de OS e DisplayServer).

2.4 Core

O núcleo (Core) da engine contém funcionalidades essenciais e estruturas de dados usadas em toda a engine, como Object e ClassDB, gerenciamento de memória, contêineres, entrada/saída de arquivos, Variant e outros utilitários.

- **Object:** Um tipo avançado de Variant. Todas as classes na engine herdam de Object. Cada classe pode definir novas propriedades, métodos ou sinais, que ficam disponíveis para todas as classes que herdam dela.
- **ClassDB:** Fornece acesso aos metadados para cada classe disponível na engine
- **Variant:** Tipo de dado mais importante utilizado no Godot. Um Variant ocupa apenas 24 bytes em plataformas de 64 bits (20 bytes em 32 bits) e pode armazenar praticamente qualquer tipo de dado da engine. Variants raramente são usados para armazenar informações por longos períodos de tempo. Em vez disso, eles sao

utilizados principalmente para comunicação, educação, serialização e, de forma geral, para movimentação de dados dentro da engine.

2.5 Main

O componente Main é responsável por inicializar e gerenciar o ciclo de vida da engine, incluindo inicialização, encerramento e o loop principal.

- **MainLoop:** A Godot Engine define o *MainLoop* como a classe base abstrata do loop principal de um projeto. Ela é herdada por *SceneTree*, que é a implementação padrão do loop de jogo utilizada nos projetos do Godot, embora também seja possível criar e usar uma subclasse própria de *MainLoop* em vez da árvore de cenas.

3. Processo de Exportação e Publicação

A Godot Engine simplifica o processo de exportação de jogos para diversas plataformas. O processo envolve a criação de presets de exportação e a instalação de templates específicos para cada plataforma
.

3.1. Exportação de Projetos

Para exportar um projeto na Godot, o desenvolvedor acessa o menu Exportar e adiciona um preset para a plataforma desejada. Cada preset contém configurações específicas para a plataforma de destino. A Godot requer a instalação de templates de exportação, que são pacotes pré-compilados para cada plataforma, e, em alguns casos, SDKs adicionais (Software Development Kits)
.

Modos de Exportação:

- **Exportar Tudo:** Cria um build jogável (executável da Godot e dados do projeto) para todos os presets definidos.
- **Exportar Projeto:** Cria um build jogável para um preset selecionado.
- **Exportar PCK/ZIP:** Exporta apenas os recursos do projeto como um pacote .pck ou .zip, sem o executável da Godot. Isso é útil para DLCs ou para usar com executáveis Godot personalizados

.

3.2. Plataformas Suportadas

A Godot suporta a exportação para uma ampla variedade de plataformas
:

- **PC:** Windows (.exe), Linux (qualquer extensão, tipicamente .x86_64), macOS (.app ou .zip).
- **Web:** HTML5 (.zip), permitindo que o jogo seja executado em navegadores que suportam WebAssembly e WebGL 2.0

.

- **Mobile:** Android (.apk), iOS (.zip).

- **Consoles:** A exportação para consoles (PlayStation, Xbox, Nintendo Switch) requer acesso aos SDKs de desenvolvimento de cada plataforma, que são fornecidos pelos fabricantes dos consoles. Isso geralmente envolve um processo de licenciamento e parceria com estúdios ou publishers

.

3.3. Publicação

Após a exportação, o processo de publicação varia de acordo com a plataforma:

- **Lojas de PC (Steam, Epic Games Store, GOG):** Envolve o upload do build exportado, metadados do jogo (descrição, capturas de tela, trailers), configuração de preços e aprovação pela plataforma.
- **Web (itch.io, sites próprios):** O build HTML5 pode ser facilmente hospedado em servidores web ou plataformas como itch.io.
- **Lojas Mobile (Google Play Store, Apple App Store):** Requer o registro como desenvolvedor, upload do .apk ou .ipa (para iOS), preenchimento de informações da loja, configuração de preços e revisão pelas respectivas lojas.
- **Consoles:** O processo é mais complexo, exigindo um acordo com o fabricante do console e a submissão do jogo para certificação e testes rigorosos antes da publicação.

4. Referências

Godot Engine. Documentação Oficial. Disponível em: https://docs.godotengine.org/